

HW5

Sri Srinivasan

July 2025

Problem 1

(a)

Analytically, for the given OLS problem $\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$, the least-squares solution is given by:

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Given $\mathbf{X} = \begin{pmatrix} 10 & 1 \end{pmatrix}$ and $\mathbf{y} = (11)$:

$$\mathbf{X}^\top \mathbf{X} = \begin{pmatrix} 10 \\ 1 \end{pmatrix} \begin{pmatrix} 10 & 1 \end{pmatrix} = \begin{pmatrix} 100 & 10 \\ 10 & 1 \end{pmatrix}$$

This matrix is singular ($\det(\mathbf{X}^\top \mathbf{X}) = 100 \times 1 - 10 \times 10 = 0$). This means there are infinitely many solutions that achieve a loss of 0 because the system $10w_1 + w_2 = 11$ is underdetermined for finding unique w_1 and w_2 .

Using "np.linalg.pinv":

$$\mathbf{w} = \text{pinv}(\mathbf{X})\mathbf{y}$$

The output from the code is:

(a)

```
w = [[1.08910891]
      [0.10891089]]
```

So, $w_1 = 1.08910891$ and $w_2 = 0.10891089$. This solution satisfies $10w_1 + w_2 = 10(1.08910891) + 0.10891089 = 10.8910891 + 0.10891089 = 11$.

(b)

The output of the previous cell for gradient descent training is:

(b) **Weights:** `[tensor([[1.0891, 0.1089]])]`

The loss curve shows a rapid decrease initially, then stabilizes at a very low value, suggesting convergence.

(c)

The output of the previous cell for the dropout least-square calculation is:

(c)

```
x = [[20  2]
      [20  0]
      [ 0  2]
      [ 0  0]]
y = [[11]
      [11]]
```

```
[11]
[11]]
w = [[0.36666667]
      [3.66666667]]
```

The new dataset $\mathbf{X}$ reflects the four equally likely input combinations after applying dropout with $p = 0.5$ and scaling by $2 = 1/(1 - 0.5)$:

- Neither dropped out: $x_{\text{original}} \times 2 = [10, 1] \times 2 = [20, 2]$
- w_2 dropped out: $[10, 0] \times 2 = [20, 0]$
- w_1 dropped out: $[0, 1] \times 2 = [0, 2]$
- Both dropped out: $[0, 0] \times 2 = [0, 0]$

The corresponding $\mathbf{y}$ values remain 11. The calculated weights are $w_1 = 0.36666667$ and $w_2 = 3.66666667$.

(d)

The output after training with gradient descent and dropout ($p = 0.5$) is:

(b) **Weights:** `[tensor([[0.4819, 3.5143]])]`

The training curve with dropout (using a batch size of 1) exhibits a highly fluctuating and noisy shape. The loss does not converge smoothly to a single value but rather jumps significantly between iterations. This is expected because with a batch size of 1 and a dropout rate of 0.5, each iteration uses a randomly masked and scaled input, leading to a very high variance in the gradients and in the loss.

The weights obtained by training with gradient descent ($w_1 = 0.4819, w_2 = 3.5143$) are not the same as those calculated using the closed-form least squares method ($w_1 = 0.36666667, w_2 = 3.66666667$). The empirical weights are close, but due to the high variance introduced by dropout with a small batch size, the training might not have fully converged to the true expected analytical solution in 1000 iterations.

(e)

The output after training with gradient descent and dropout ($p = 0.5$) with a larger batch size (1024) is:

(b) **Weights:** `[tensor([[0.3787, 3.4829]])]`

The loss curve with a larger batch size (1024) is smoother and decreases more consistently compared to the loss curve with a batch size of 1. While there are still some fluctuations due to the stochastic nature of dropout, they are much smaller in magnitude. This is because using a larger batch size effectively averages out the random dropout masks across many samples within each iteration, providing a more stable estimate of the gradient.

The trained weights with the larger batch size ($w_1 = 0.3787, w_2 = 3.4829$) are much closer to the analytically calculated least-squares weights ($w_1 = 0.36666667, w_2 = 3.66666667$) than the weights obtained with a batch size of 1. This demonstrates that larger batch sizes reduce the variance in the gradients caused by dropout, leading to a more stable training process and convergence closer to the expected solution.

(f)

The plots show that both the analytical and empirical solutions for w_1 , w_2 , and the loss generally change smoothly as the dropout rate p increases. The empirical results (scatter points) align well with the analytical curves, especially for the larger dropout rates.

There is a noticeable discontinuity when the dropout rate is 0 ($p = 0$).

- When $p = 0$, dropout is effectively turned off. The input to the linear layer is simply $\mathbf{X} = \begin{pmatrix} 10 & 1 \end{pmatrix}$. As derived in part (a), the least-squares solution using the pseudoinverse yields $w_1 \approx 1.089$ and $w_2 \approx 0.109$.

- When $p > 0$, even a very small p , dropout is active. The four distinct inputs (scaled) are considered with their respective probabilities. The analytical solution for weighted least squares takes into account these probabilities. For any $p > 0$, the structure of the $\mathbf{X}$ matrix in the weighted least squares effectively changes to incorporate these varied inputs and their likelihoods. Specifically, the X matrix for the weighted least squares includes the zero input $[0, 0]$ with probability p^2 , and the scaling factor $1/(1-p)$ is applied to non-zero elements. This fundamentally alters the problem's effective data distribution and thus its least-squares solution.

The discontinuity at $p = 0$ arises because dropout fundamentally changes the problem formulation from a single input to a distribution of inputs (even if the expected input remains the same for the non-weighted case). At $p = 0$, there is no randomness and no scaling, so the problem is the original least-squares problem with a single data point. For any $p > 0$, the problem becomes a weighted least squares problem over the four possible outcomes, leading to a different solution space, especially for the w_1 and w_2 values. The w_1 and w_2 curves for $p > 0$ show the solution for the weighted average of the four possible inputs, which is different from the $p = 0$ case where only one input type is considered.

(g)

The plot shows the trained weights w_1 and w_2 for both SGD and Adam optimizers at dropout rates of 0 and 0.5.

- When dropout rate is 0:
 - SGD and Adam both converge to approximately the same weights: $w_1 \approx 1.089$ and $w_2 \approx 0.109$. This is the least-squares solution for the original problem without dropout. This indicates that both optimizers can find the optimal solution in this simple, convex setting.
- When dropout rate is 0.5:
 - SGD converges to $w_1 \approx 0.379$ and $w_2 \approx 3.483$. These values are very close to the analytical solution found in Part (e) ($w_1 = 0.36666667, w_2 = 3.66666667$).
 - Adam, however, converges to significantly different weights: $w_1 \approx 0.1$ and $w_2 \approx 3.667$. It seems that for w_2 , Adam converges to the least square value from (c) while w_1 has a smaller value.

The solutions change with the optimizer when dropout is applied because Adam (and other adaptive optimizers) behaves differently than SGD in the presence of high variance gradients, which are introduced by dropout.

- SGD uses a constant learning rate for all parameters. With dropout, the gradients are stochastic, and SGD might "bounce around" more, eventually averaging out to the expected gradient direction, especially with a large batch size.
- Adam adapts the learning rate for each parameter based on the past squared gradients (first and second moments). Adam might be more aggressive in some directions and less in others, leading to a different final solution compared to SGD, especially if the loss landscape becomes less "smooth" due to the high variance.

In this specific problem, it seems Adam's adaptive learning rates, while generally beneficial for faster convergence in complex deep learning, might lead to a slightly different final solution in this simplified, high-variance dropout scenario compared to SGD, even with large batch sizes. This highlights that while dropout introduces stochasticity, the optimizer's mechanism for handling these stochastic gradients can influence the final converged weights.

(h)

Model without Dropout (dropout_rate=0):

```

Train Epoch: 1 of 10 Train Loss: 1.487, Val Loss: 1.258129, Val Accuracy: 59.000000
Train Epoch: 2 of 10 Train Loss: 0.817, Val Loss: 0.547143, Val Accuracy: 84.000000
Train Epoch: 3 of 10 Train Loss: 0.395, Val Loss: 0.233075, Val Accuracy: 95.000000
Train Epoch: 4 of 10 Train Loss: 0.170, Val Loss: 0.088612, Val Accuracy: 98.000000
Train Epoch: 5 of 10 Train Loss: 0.076, Val Loss: 0.038144, Val Accuracy: 99.000000
Train Epoch: 6 of 10 Train Loss: 0.039, Val Loss: 0.019159, Val Accuracy: 100.000000
Train Epoch: 7 of 10 Train Loss: 0.021, Val Loss: 0.009951, Val Accuracy: 100.000000
Train Epoch: 8 of 10 Train Loss: 0.012, Val Loss: 0.005574, Val Accuracy: 100.000000
Train Epoch: 9 of 10 Train Loss: 0.008, Val Loss: 0.003362, Val Accuracy: 100.000000
Train Epoch: 10 of 10 Train Loss: 0.006, Val Loss: 0.002206, Val Accuracy: 100.000000
Accuracy on clean data: 6867/10000 (69%)
Accuracy on data with cheating feature: 10000/10000 (100%)

```

Model with Dropout (dropout_rate=0.75):

```

Train Epoch: 1 of 10 Train Loss: 2.181, Val Loss: 2.170965, Val Accuracy: 15.000000
Train Epoch: 2 of 10 Train Loss: 2.030, Val Loss: 1.932822, Val Accuracy: 30.000000
Train Epoch: 3 of 10 Train Loss: 1.839, Val Loss: 1.706173, Val Accuracy: 43.000000
Train Epoch: 4 of 10 Train Loss: 1.670, Val Loss: 1.503460, Val Accuracy: 51.000000
Train Epoch: 5 of 10 Train Loss: 1.528, Val Loss: 1.341398, Val Accuracy: 57.000000
Train Epoch: 6 of 10 Train Loss: 1.411, Val Loss: 1.222716, Val Accuracy: 61.000000
Train Epoch: 7 of 10 Train Loss: 1.320, Val Loss: 1.127815, Val Accuracy: 64.000000
Train Epoch: 8 of 10 Train Loss: 1.250, Val Loss: 1.050302, Val Accuracy: 67.000000
Train Epoch: 9 of 10 Train Loss: 1.189, Val Loss: 0.990422, Val Accuracy: 69.000000
Train Epoch: 10 of 10 Train Loss: 1.139, Val Loss: 0.941641, Val Accuracy: 71.000000
Accuracy on clean data: 6991/10000 (70%)
Accuracy on data with cheating feature: 7064/10000 (71%)

```

When no dropout is used (dropout_rate=0), the model achieves near-perfect accuracy (100%) on data with the cheating feature. However, its accuracy drops significantly to 69% on clean data (without the cheating feature). This indicates that the model without dropout heavily relies on the cheating feature to make predictions. It has learned to "read" the encoded label in the corner rather than learning the actual visual features of the CIFAR10 images.

When dropout is applied with a high rate (dropout_rate=0.75), the model's behavior changes dramatically:

- Its accuracy on data with the cheating feature drops significantly from 100% to 71%.
- Its accuracy on clean data slightly increases from 69% to 70%.

The model with dropout (0.75) does slightly better on clean data (70% accuracy) compared to the model without dropout (69% accuracy). This indicates that dropout helps the model generalize better to data where the extra feature is absent.

Problem 2

(a)

A potential reason for this observation is vanishing/exploding gradients, which may make it difficult to train the earlier layers well, leading to underfitting. Another possible reason is poor initialization, since without this, very deep networks may fail to propagate gradients effectively. Changes that could help mitigate the problem are using residual connections and batch normalization.

(b)

The model that does not pass the test is the one using layer normalization. This is because layer normalization and batch normalization are fundamentally different: batchnorm normalizes across the batch dimension,

whereas layernorm normalizes across the feature dimension for each individual sample. Therefore, changing the batch size affects batchnorm but not layernorm. Therefore, the outputs of the batchnorm model are sensitive to effective batch size, whereas layernorm model's outputs are not. This means only the batchnorm model will fail the equivalence test across the different combinations.

(c)

The optimizer is being stepped within the loop over the training set, which results in a parameter update after every mini-batch. Also, the gradients are not being zeroed inside the loop, so gradients keep accumulating across all batches. The fix is to call `optimizer.zero_grad()` inside the loop to reset gradients before each mini-batch.

```
model.train()
for (inputs, labels) in training_set:
    optimizer.zero_grad()
    predictions = model(inputs)
    loss = loss_fn(predictions, labels)
    loss.backward()
    optimizer.step()
```

Problem 6

(a)

No, K-Means likely did not work effectively on this dataset. K-Means operates under the assumption that clusters are spherical and have similar variances. It minimizes the sum of squared distances between data points and their assigned cluster centroids. K-Means, using Euclidean distance to centroids, will struggle to correctly identify and separate these non-spherical, linearly-structured clusters. Instead, it will tend to cut across the elongated shapes, resulting in cluster assignments that do not reflect the true structure of the data.

(c)

The graph is undirected.

This is because the adjacency matrix A is symmetric, which is a characteristic of an undirected graph. The formula for the edge weight between nodes i and j is given by:

$$A_{i,j} = e^{\gamma \|x_i - x_j\|^2}$$

Since the Euclidean distance is symmetric, i.e., $\|x_i - x_j\| = \|x_j - x_i\|$, it follows that:

$$A_{i,j} = e^{\gamma \|x_i - x_j\|^2} = e^{\gamma \|x_j - x_i\|^2} = A_{j,i}$$

Thus, the matrix A is symmetric ($A = A^T$), indicating that the relationship (edge) between node i and node j is the same as the relationship between node j and node i .

(f)

We can see a significant improvement over the initial K-Means application on the raw data. The algorithm successfully identifies and separates the two primary elongated, non-spherical clusters. The data points within each elongated structure are predominantly assigned to the same cluster, indicated by a consistent color. A third cluster is also formed, which might represent a sub-group within one of the main elongated structures or a small, distinct region. Yes, the approach worked effectively for identifying the two main elongated clusters.

(g)

The clustering results are effective. The two elongated clusters are clearly and accurately segmented, with points belonging to the same elongated structure grouped together. The third cluster is also distinctly formed. The reason for its success is the same as in part (f). By projecting the data onto the space spanned by these principal eigenvectors, the algorithm creates a new representation where the clusters, even if non-linear or non-spherical in the original space, become separable by a simple distance-based algorithm like K-Means.

Problem 4

(a)

Suppose $\mathbf{x} \sim p_\mu(\mathbf{x})$ where p_μ is a multivariate Gaussian:

$$p_\mu(\mathbf{x}) = (2\pi\sigma^2)^{-n/2} \exp\left(-\frac{\|\mathbf{x} - \mu\|_2^2}{2\sigma^2}\right).$$

Then the gradient is given by the score function (likelihood-ratio) trick:

$$\nabla_\mu \mathcal{F}(\mu) = \mathbb{E}_{\mathbf{x} \sim p_\mu} [f(\mathbf{x}) \cdot \nabla_\mu \log p_\mu(\mathbf{x})].$$

Compute:

$$\nabla_\mu \log p_\mu(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sigma^2}.$$

Therefore,

$$\nabla_\mu \mathcal{F}(\mu) = \mathbb{E}_{\mathbf{x} \sim p_\mu} \left[f(\mathbf{x}) \cdot \frac{\mathbf{x} - \mu}{\sigma^2} \right].$$

(b)

Use the change of variables $\mathbf{x} = \mu + \sigma\mathbf{z}$, with $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$. Then:

$$\mathcal{F}(\mu) = \mathbb{E}_{\mathbf{x} \sim p_\mu} [f(\mathbf{x})] = \mathbb{E}_{\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})} [f(\mu + \sigma\mathbf{z})].$$

(c)

Taking the gradient with respect to μ under the reparameterization:

$$\nabla_\mu \mathcal{F}(\mu) = \nabla_\mu \mathbb{E}_{\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})} [f(\mu + \sigma\mathbf{z})] = \mathbb{E}_{\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})} [\nabla_\mu f(\mu + \sigma\mathbf{z})].$$

Using the chain rule:

$$\nabla_\mu f(\mu + \sigma\mathbf{z}) = \nabla f(\mu + \sigma\mathbf{z}),$$

so:

$$\nabla_\mu \mathcal{F}(\mu) = \mathbb{E}_{\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})} [\nabla f(\mu + \sigma\mathbf{z})].$$

(d)

In the general case, we can use the score function estimator again:

$$\nabla_\theta \mathcal{F}(\theta) = \nabla_\theta \mathbb{E}_{\mathbf{x} \sim p_\theta} [f(\mathbf{x})] = \mathbb{E}_{\mathbf{x} \sim p_\theta} [f(\mathbf{x}) \cdot \nabla_\theta \log p_\theta(\mathbf{x})].$$

(e)

Suppose $\mathbf{x} = g(\mathbf{z}, \theta)$, where $\mathbf{z} \sim p(\mathbf{z})$, and g is a differentiable function such that $\mathbf{x} \sim p_\theta$. Then:

$$\mathcal{F}(\theta) = \mathbb{E}_{\mathbf{x} \sim p_\theta} [f(\mathbf{x})] = \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [f(g(\mathbf{z}, \theta))].$$

Taking the gradient:

$$\nabla_\theta \mathcal{F}(\theta) = \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [\nabla_\theta f(g(\mathbf{z}, \theta))].$$

Using the chain rule:

$$\nabla_\theta f(g(\mathbf{z}, \theta)) = \nabla g(\mathbf{z}, \theta)^T \cdot \nabla f(g(\mathbf{z}, \theta)),$$

so:

$$\nabla_\theta \mathcal{F}(\theta) = \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [\nabla g(\mathbf{z}, \theta)^T \cdot \nabla f(g(\mathbf{z}, \theta))].$$

Problem 5

(a)

We are asked to show that:

$$L^{SymNorm} = I - A^{SymNorm}$$

can also be written as:

$$L^{SymNorm} = D^{-1/2} L D^{-1/2}$$

Since $L = D - A$, we substitute:

$$L^{SymNorm} = D^{-1/2} (D - A) D^{-1/2} = D^{-1/2} D D^{-1/2} - D^{-1/2} A D^{-1/2} = I - A^{SymNorm}$$

Thus, we have proven the desired expression.

(b)

For the undirected graph in Figure 4, we have 5 nodes. The adjacency matrix A , degree matrix D , and symmetrically normalized adjacency matrix $A^{SymNorm}$ are:

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$D = \begin{bmatrix} 3 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 & 2 \end{bmatrix}$$

Then:

$$A^{SymNorm} = D^{-1/2} A D^{-1/2}$$

Let $D^{-1/2} = \text{diag}(1/\sqrt{3}, 1/\sqrt{3}, 1/\sqrt{2}, 1/2, 1/\sqrt{2})$. Multiply as:

$$A_{i,j}^{SymNorm} = \frac{A_{i,j}}{\sqrt{D_{ii} D_{jj}}}$$

So for example:

$$A_{1,2}^{SymNorm} = \frac{1}{\sqrt{3 \cdot 3}} = \frac{1}{3}, \quad A_{1,4}^{SymNorm} = \frac{1}{\sqrt{3 \cdot 4}} = \frac{1}{2\sqrt{3}}, \text{ etc.}$$

(c)

Using the formula:

$$L^{SymNorm} = I - A^{SymNorm},$$

we subtract each element in $A^{SymNorm}$ from the identity matrix.

(d)

Compute A^2 and A^3 using matrix multiplication:

$$A^2 = A \cdot A, \quad A^3 = A \cdot A^2$$

This gives counts of 2-hop and 3-hop walks between nodes in the graph.

(e)

For the directed graph in Figure 5, write the unnormalized adjacency matrix A . Each entry $A_{i,j} = 1$ if there is an arrow from node i to node j .

(f)

We must compute: In-degree matrix D_{in} : count of incoming edges per node and out-degree matrix D_{out} : count of outgoing edges per node.

Each is diagonal, with $D_{in}(j,j) = \sum_i A_{i,j}$, $D_{out}(i,i) = \sum_j A_{i,j}$.

(g)

The symmetrically normalized Laplacian for a directed graph is:

$$L^{SymNorm} = I - D_{out}^{-1/2} A D_{in}^{-1/2}$$

assuming we use both in- and out-degree matrices for normalization.